

Support AI Ticket Management Agent

Description · System Architecture · Detailed Process | React + Django + MongoDB + JWT + NLP

1. Project Description

What is this project?

A customer-support ticketing platform where an **AI layer automatically reads every incoming ticket**, decides which category it belongs to, predicts how urgent it is, and suggests a reply based on similar problems solved before. Support agents get a smart dashboard where the most urgent tickets automatically rise to the top — so they triage in **seconds instead of minutes**.

The problem it solves

- **Slow triage:** today, agents read hundreds of tickets one by one and manually sort them.
- **Urgent tickets get buried:** “my money got debited twice!” waits in the same line as a minor question.
- **Misrouting:** billing issues land with technical agents and bounce between teams.
- **Repeated work:** agents keep typing the same answers to the same problems.
- **SLA breaches:** without automatic urgency detection, response deadlines get missed.

Who uses it? (three roles)

- **Customer:** submits tickets (subject + description), tracks the status of their own tickets only.
- **Agent:** works the priority-sorted queue, uses AI-suggested replies, moves tickets through the lifecycle (Open → In Progress → Resolved → Closed).
- **Admin:** manages users and monitors the overall queue and SLA performance.

Where is the “intelligence”? (honest scope)

This is **NOT a ChatGPT-style chatbot**. It is three focused, production-style AI capabilities:

- **Category classification:** a text classifier (TF-IDF + Naive Bayes / SVM) reads the ticket and predicts Billing / Technical / Account / General.
- **Priority prediction:** urgency keywords + sentiment analysis detect angry or urgent language and push the ticket to HIGH priority.
- **Suggested reply (retrieval-based):** cosine similarity finds the most similar past *resolved* ticket and shows the agent how it was solved.

This scope is realistic, demonstrable, and matches how real support platforms actually use AI.

2. System Architecture



Why this layered design? (the key teaching point)

- **Views never contain business or NLP logic.** The API layer checks security and validates input, then calls the service layer.
- **The NLP engine is isolated** — the classifier and its TF-IDF vectorizer are saved and loaded together, and can be retrained without touching the website.
- **Graceful failure:** if the model ever errors, the ticket is still created with safe defaults (General / Medium) — AI must never block the business.
- **Strict data isolation:** a customer's query is always filtered to their own tickets — they can never see another customer's data.

Security built into the architecture

- Passwords stored only as hashes
- JWT access + refresh tokens with expiry
- CORS locked to the known frontend origin
- 3-role permission model on every endpoint
- Request throttling against abuse

Data flow in one line

Customer submits ticket → JWT-secured API → serializer validates → service calls NLP (category + priority) → ticket saved in MongoDB → appears in agent's priority queue → agent opens it → similarity search suggests a reply → agent resolves → history logged.

3. Detailed Process — Step by Step

1

Customer submits a ticket

After a secure JWT login, the customer fills subject + description (optionally attaching a screenshot). The React form validates first; the Django serializer validates again on the server.

2

AI classifies the category — instantly, at creation time

The service layer sends the ticket text to the NLP engine. **TF-IDF** converts the words into numbers; the trained classifier (**Naive Bayes / SVM**) predicts: Billing, Technical, Account, or General. If the model fails for any reason, the ticket safely defaults to General — creation never breaks.

3

AI predicts the priority

Urgency keywords (“urgent”, “not working”, “debited twice”) and **sentiment analysis** (angry or highly negative tone) push the ticket to HIGH priority. The SLA deadline is then set automatically from the priority — e.g. HIGH must be answered fastest.

4

Ticket lands in the smart agent queue

The agent dashboard shows the queue **sorted by predicted priority**, with filters by category and status, and a live SLA countdown per ticket. Urgent problems are on top the moment they arrive — no human sorting needed.

5

AI suggests a reply from past solved tickets

When the agent opens a ticket, **cosine similarity** compares its text against all past *resolved* tickets and shows the closest match with its resolution and a similarity score — “a similar problem was solved like this before.” The system also records whether the agent used the suggestion, which measures how useful the AI is.

6

Agent resolves — the full lifecycle is tracked

The ticket moves Open → In Progress → Resolved → Closed. Every status change is written to **ticket_history** with who changed it and when — a complete audit trail, exactly like enterprise support tools.

7

The system keeps improving

Every resolved ticket becomes new training data. Retraining the classifier on the growing history makes category and priority predictions more accurate over time, and gives the similarity search more solved examples to suggest from — **the platform gets smarter with every ticket it handles.**